L2L: Logic to Layout Exploration of Standard Cell Library Design

Byeonggon Kang
University of California, San Diego
La Jolla, California, USA
b8kang@ucsd.edu

Alan Mishchenko
University of California, Berkeley
Berkeley, California, USA
alanmi@berkeley.edu

Masahiro Fujita
The University of Tokyo and AIST
Tokyo, Japan
fujita@ee.t.u-tokyo.ac.jp

Bill Lin
University of California, San Diego
La Jolla, California, USA
billlin@eng.ucsd.edu

Chung-Kuan Cheng
University of California, San Diego
La Jolla, California, USA
ckcheng@ucsd.edu

Abstract

We present L2L, exploring end-to-end design optimization from logic to layout. The L2L framework includes logic, circuit, and layout stages. Logic exploration introduces two-stage transistor network synthesis. At the circuit stage, we propose two SMT models for transistor network synthesis: *Connected-Diffusion* (arbitrary, stack-height-bounded, multi-solution) and *Grid-Scaffold* (*series-parallel*). At the layout stage, an ILP-based flow co-optimizes circuit topology and double-height layouts, producing two 3-input P-class libraries (area- and metal-optimized). Block-level experiments demonstrate that full inclusion of 3-input P-class functions achieves up to 1.3% lower power, 5.6% higher performance, and 4.1% smaller area, outperforming the best prior baseline (average improvements: 0.8%, 0.6%, and 2.6%, respectively).

CCS Concepts

• **Hardware → Standard Cell Library**.

Keywords

Standard cell, transistor network synthesis, cell layout, SMT, ILP

ACM Reference Format:
Byeonggon Kang, Alan Mishchenko, Masahiro Fujita, Bill Lin, and Chung-Kuan Cheng. 2026. L2L: Logic to Layout Exploration of Standard Cell Library Design. In *63rd ACM/IEEE Design Automation Conference (DAC '26), July 26–29, 2026, Long Beach, CA, USA.* ACM, New York, NY, USA, 7 pages. https://doi.org/10.1145/3770743.3804220

1 Introduction

Standard cells are the essential building blocks of digital design. Integrating the design flow from logic to layout is critical to improving the quality of the cell library.

Layout automation for standard cells has been studied extensively [6]. To improve cell layout quality, prior work highlights key aspects such as pin accessibility [9, 16], simultaneous placement and routing optimization [10], and cell architecture optimization [7]. However, beyond layout optimizations, the underlying circuit realizations and the richness of the cell library fundamentally bound the achievable block-level power, performance, and area (PPA). **Transistor network synthesis** [12, 17, 19] derives a transistor-level circuit from a given Boolean function. Possani et al. [12] employ a graph-based kernel-identification and network composition framework to effectively reduce transistor count for complex gates and verify stack-height bounds to prevent performance degradation; however, the search is heuristic and does not exhaustively explore the solution space. MiniTNtk [19] adopts an exact, incremental SAT-based approach using a generalized switching graph, but checks stack height after the network is synthesized and restricts single-stage synthesis to gate inputs from primaries and their inversions, limiting fully minimized networks. MuSTNet [17] further advances exact multi-stage transistor network synthesis by modeling a MOS–net bipartite graph with multi-commodity-flow-based function verification and adding placement-aware constraints that consider cell area. However, stack height is not explicitly constrained (potential performance degradation), and the model does not cover design rules, pin access, and internal net routing, making precise area estimation challenging.

To bridge the gap between transistor network synthesis and layout automation, we propose **L2L** (Logic-to-Layout), an end-to-end framework, illustrated in Figure 1.

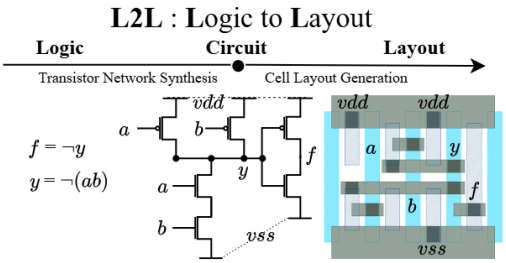

Figure 1: L2L: Logic-to-Layout exploration framework.

Our contributions are as follows:

- **L2L framework.** We construct an end-to-end L2L framework. This work focuses on the full exploration of 3-input P-class functions as a first step toward scalable logic-to-layout exploration.
- **Logic Exploration.** We introduce two-stage transistor network synthesis using a permutation-closed input pool.
- **Circuit Exploration.** We propose two SMT models for transistor network synthesis: 1) *Connected-Diffusion* for arbitrary topology **integrates an explicit stack-height bound** and **collects multiple cost-optimal solutions**, and 2) *Grid-Scaffold* for *series-parallel*. We derive NMOS from each PMOS solution using a **planar dual** and **complement-polarity transform**, reducing search costs and enhancing stack balance/layout quality.
- **Layout Exploration.** We generate standard cell layouts with circuit-topology optimization and double-height optimization.
- **3–input P-class cell libraries.** We release two open-source libraries [1] covering all 3-input P-class functions: (i) an *Area-Optimized* set ($L2L_{AO}$) and (ii) a *Metal-usage-Optimized* set ($L2L_{MO}$).
- **Block–level evaluation.** Adding the full 3-input P-class functions yields up to 1.3% lower power, 5.6% higher performance, and 4.1% smaller area (average improvements: 0.8%, 0.6%, and 2.6%, respectively).

This work is licensed under a Creative Commons Attribution 4.0 International License.
DAC '26, Long Beach, CA, USA
© 2026 Copyright held by the owner/author(s).
ACM ISBN 979-8-4007-2254-7/2026/07
https://doi.org/10.1145/3770743.3804220

2 Preliminary

We provide essential background for our study: logic in Sec. 2.1, transistor network in Sec. 2.2 and cell layout generation in Sec. 2.3.

2.1 Boolean Function Equivalence Class

Boolean functions form equivalence classes under common transformations [8]: NPN-classes allow input/output negation and input permutation; P-classes allow input permutation only. Thus, every P-class is a subset of an NPN-class. P-class representatives can yield a smaller number of contact poly pitch (#CPP, the basic area unit of layout size) for the same transistor count, making them more layout-efficient than NPN representatives. Figure 2 depicts two

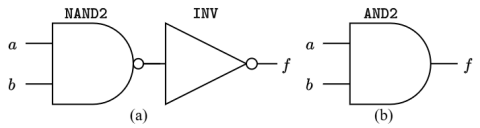

Figure 2: Two ways to implement AND2.

ways to implement AND2: (a) using NAND2 followed by an INV, and (b) using a dedicated AND2 cell. Both use 6 transistors, but #CPP differs: (a) 3 CPP (NAND2) + 2 CPP (INV) = 5 CPP, whereas (b) fits in 4 CPP—about a 20% area reduction. Therefore, we aim to develop a cell library that includes all 3-input P-class functions.

2.2 Transistor Network Topologies

Based on structural properties, transistor networks can be classified as *series–parallel* or non–series–parallel (NSP) [13], and as planar or non-planar. NSP networks introduce cross-connections between intermediate nodes in series chains, enabling reductions in transistor count; however, they allow many transistors connected on shared nets, which increases routing congestion. In contrast, *series–parallel* networks have an advantage for diffusion sharing and simpler net connections, making them more layout-friendly; however, they may require more transistors. In this work, we synthesize both *series–parallel* and arbitrary topologies, generate layouts for each, and choose the design that minimizes #CPP or metal usage.

2.3 Cell Layout Automation Framework

Automatic cell layout generation is critical to the enablement of standard cell libraries. A variety of engines—spanning reinforcement learning [14], search–tree heuristics [2], ILP [5], and SMT [10]—map a given circuit to a layout; in this work, we benchmark SO3-Cell [5]. This choice is motivated by three needs. First, simultaneous optimization of circuit and layout: to obtain novel layouts, the transistor network derived from logic must undergo circuit–topology optimization alongside placement/routing. Second, compatibility with advanced–node standard–cell architectures [4] (e.g., M0 4-track with unidirectional routing). Third, the availability of an open-source, end-to-end flow from cell layout to block-level design.

3 Logic: Two-Stage Synthesis

In conventional "one-stage" transistor network synthesis [19], gate controls are taken from the primaries and their inversions, $X \cup \bar{X}$. Since realizing $\bar{X}$ requires inverters—i.e., an additional physical stage—this setting is logically two-stage. We define the one–stage gate set O as the Boolean functions implementable by a single PMOS/NMOS (P/N) network (pull-up/pull-down producing one output) whose gate terminals draw only from the primaries $X = \{x_1, \ldots, x_i\}$. Two–stage synthesis composes a second network whose gate controls may use X and any first-stage function from a permutation-closed pool O^p built from O. Let $i = |X|$ and let m be the arity of $o \in O$. To remove pin-ordering bias, for each o we enumerate all $\binom{i}{m}$ choices of m primaries and all $m!$ input permutations, derive the corresponding i-input truth-table signatures, and deduplicate them to retain one canonical representative per distinct function. For $X = \{a, b, c\}$, $\texttt{NAND2}_{a,b}$, $\texttt{NAND2}_{b,c}$, and $\texttt{NAND2}_{c,a}$ realize different functions of (a, b, c) and thus all appear in O^p (one canonical representative per distinct signature).

4 Circuit: Transistor Network Synthesis

We propose two SMT models for two-stage PMOS transistor network synthesis: 1) *Connected-Diffusion* for arbitrary topology, and 2) *Grid-Scaffold* for *series–parallel*. The NMOS pull-down is then obtained via complement-pair derivation, yielding the final P/N circuit.

4.1 Objective Function

Gate-control signals have different physical costs in our two-stage synthesis (e.g., $\bar{x}$ requires an inverter). We assign a signal cost c_p and charge it once if p is used on any transistor gate: $c_x = 0$ for $x \in X$, $c_{\bar{x}} = 1$ for $\bar{x} \in \bar{X}$, and c_p equals half of the minimum transistor count for $p \in O^p$. We capture this cost using

$$gateCost = \sum_{p \in \text{input pool}} \text{ite}\left(\bigvee_{t \in T} (g_t = p),\ c_p,\ 0 \right), \tag{1}$$

where g_t is the gate signal of transistor t. We minimize $gateCost$.

4.2 *Connected-Diffusion* for Arbitrary Topology

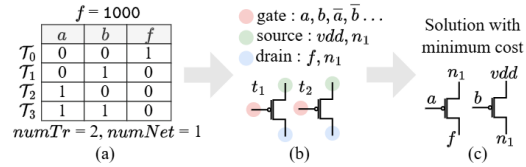

Figure 3: *Connected-Diffusion* synthesis example. (a) Inputs. (b) Net classification. (c) The synthesized PMOS network.

We present *Connected–Diffusion*, an SMT-based exploration of arbitrary topologies (NOR2 example in Fig. 3). At the level of basic device modeling, our SMT formulation follows the previous SAT-based exact-synthesis setting [17, 19]: for each truth-table row, input assignments determine the ON/OFF state of devices, and the solution must satisfy all rows simultaneously. The differences are twofold. 1) Algorithm 1 performs a nested sweep over (*numTr*, *numNet*) (Sec. 4.2.1), tightening a moving cost bound (*budget*) to prune the search space and to gather cost-optimal solutions, stratified by the numbers of transistors and nets. 2) We propose a stepwise *vdd*-propagation model that incorporates an explicit stack-height constraint (Sec. 4.2.2). For rows with f=1, *vdd* must reach the output within M (maximum stack height) propagation steps. For rows with f=0, we enforce satisfiability under *vdd*=true, f=false to forbid any conduction regardless of stack height.

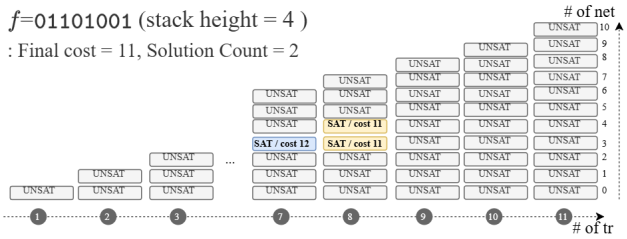

Figure 4: Search trace of Algorithm 1 for f = 01101001.

4.2.1 ***Search-Space Sweep and Pruning.*** Figure 4 shows the search trace of Algorithm 1 for f = `01101001`: the nested sweep visits ($numTr$, $numNet$) from $(1, 0)$ up to ($maxTr$, $maxTr$-1) — from all-parallel to all-series. At $(7, 3)$, a feasible network with cost 12 is found, tightening $maxTr$ to 12. Improved solutions of cost 11 at $(8, 3)$ and $(8, 4)$ update $minCost$, replace the solution set, and reduce $maxTr$ to 11. The sweep continues up to the current $maxTr$, guaranteeing optimality while retaining all minimal networks.

Algorithm 1 *Connected-Diffusion* synthesis flow

```
Input: Set of i-input P-class functions F_P; maximum stack height M.
Output: solutions and minCost for each f ∈ F_P.
for each f ∈ F_P do
    numTr ← 1; maxTr ← 18; minCost ← ∞; solutions ← ∅
    while numTr ≤ maxTr do                          ▷ loop over transistor count
        for numNet ← 0 to numTr − 1 do              ▷ loop over net count
            budget ← maxTr − numTr                  ▷ gateCost ≤ budget
            (result, network, c_g^⋆) ← FindTnSMT(f, numTr, numNet, budget, M)
            if result = SAT then
                finalCost ← c_g^⋆ + numTr
                if finalCost < minCost then
                    minCost, maxTr ← finalCost; solutions ← network
                else if finalCost = minCost then
                    Append(network, solutions)
                end if
            end if
        end for
        numTr ← numTr + 1
    end while
    save (solutions, minCost) for f
end for
```

4.2.2 ***Propagation Model with Stack-height Constraint.*** Conventional flow [19] checks the stack height after synthesis, wasting solver effort on candidates that are later discarded by the stack-height bound. We embed the stack-height constraint directly into synthesis via a stepwise vdd-propagation model bounded by M, ensuring that all synthesized networks inherently satisfy the stack-height limit. We first define sets and variables for the model:

- $N = \{vdd, f, n_1, n_2, \ldots\}$: set of nets.
- $T = \{t_1, t_2, \ldots\}$: set of transistors.
- $s \in \{0, 1, \ldots, M\}$: propagation step index bounded by M.
- $k \in \{0, 1, \ldots, 2^i\text{-}1\}$: row index of the i-input truth table.
- $\mathcal{T}_k(n) \in \{0, 1\}$: expected output of net n at truth–table row k.
- $o_{t,k} \in \{0, 1\}$: 1 if transistor t is ON in truth–table row k.
- $c^{(s)}_{n,k} \in \{0, 1\}$: 1 if net n is connected to vdd by step s in row k.

We encode stepwise vdd recursive propagation as

$$c^{(0)}_{n,k} = \begin{cases} 1 & \text{if } n = vdd, \\ 0 & \text{otherwise,} \end{cases} \qquad c^{(s+1)}_{n,k} = c^{(s)}_{n,k} \vee \bigvee_{t \in T_n} \left(o_{t,k} \wedge c^{(s)}_{n'_t,k}\right), \quad 1 \le s \le M,$$

where T_n is the set of transistors incident on net n and n'_t denotes the other terminal of transistor $t \in T_n$. A net n is connected at step s+1 if: (i) it was already connected at step s, or (ii) it becomes newly connected via a conducting transistor. Finally, we enforce the stack–height bound by requiring that, at step M, the output net f matches the expected truth–table value for every row k:

$$c^{(M)}_{f,k} = \mathcal{T}_k(f), \quad \forall k.$$

For rows with $T_k(f) = 0$, we skip propagation and use a switch-based cut: fix vdd=`true` and f=`false`; for every ON transistor, equate its source and drain (S/D). The row k is satisfied iff vdd is not identified with f, which guarantees always 0 regardless of maximum stack height M.

4.3 *Grid-Scaffold* for *Series-Parallel*

We explore only *series–parallel* topologies via the *Grid–Scaffold* shown in Fig. 5. In this section, we define the variables and constraints specific to the *Grid-Scaffold*. The placement variable $x_{p,v}$ indicates which signal $p \in \left(X \cup O^p \cup \{\texttt{short}\}\right)$ is placed at site v. For each truth-table row k, the edge flow variables $y_{k,e}$ propagate a binary flow from vdd to f.

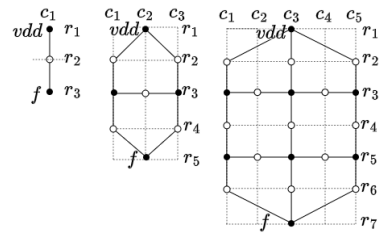

Figure 5: *Grid-Scaffold* for *series-parallel* only synthesis.

4.3.1 ***Source and sink.*** The source vdd injects flow, while the sink f receives it for function verification.

$$\bigwedge_{e \in e^{\text{out}}_k(\text{vdd})} y_{k,e}, \qquad \bigvee_{e \in e^{\text{in}}_k(f)} y_{k,e} = T_k(f).$$

4.3.2 ***Node classes and conduction rules.*** Nodes v are typed by grid parity. We define the in and out flow of v as follows:

$$\text{In}_k(v) = \bigvee_{e \in e^{\text{in}}_k(v)} y_{k,e}, \quad \text{Out}_k(v) = \bigvee_{e \in e^{\text{out}}_k(v)} y_{k,e}$$

Gate sites (odd column / even row; input pool or `short` may be placed). A gate node v is ON in row k iff a placed input evaluates to 0 or a `short` is placed. With predicate $\text{On}_k(v)$:

$$\text{Out}_k(v) \;\Leftrightarrow\; \text{On}_k(v) \wedge \text{In}_k(v).$$

That is, *on* / `short` ⇒ in = out; *off* / unplaced ⇒ out = 0.
Short-only sites (even column / odd row). Only the left/right horizontal edges are relevant. If no placement, no constraint. If `short` is placed at v, the left/right edges must agree:

$$x_{\texttt{short},v} \;\Rightarrow\; \left(y_{k,e_L(v)} \;\Leftrightarrow\; y_{k,e_R(v)}\right).$$

S/D nodes (odd column / odd row). Let $e^{\uparrow}(v)$ be the top incoming edge, and $e^{\downarrow}(v)$, $e_L(v)$, $e_R(v)$ be the bottom, left, and right edges.

$$y_{i,e^{\uparrow}(v)} \;\Rightarrow\; \left(y_{i,e_L(v)} \wedge y_{i,e_R(v)} \wedge y_{i,e^{\downarrow}(v)}\right).$$

$$\neg y_{k,e^{\uparrow}(v)} \;\Rightarrow\; \left(y_{k,e_L(v)} \Leftrightarrow y_{k,e_R(v)}\right) \wedge \left(y_{k,e_R(v)} \Leftrightarrow y_{k,e^{\downarrow}(v)}\right). \quad (2)$$

Thus, if the top in-edge is 1, all incident edges are 1 as shown in Fig. 6 (a); if the top is 0, left/right/bottom share the same value (all 1 or all 0). Eq. (2) permits inflow from other sources, such as Fig. 6 (b), but also allows floating states as shown in Fig. 6 (c).

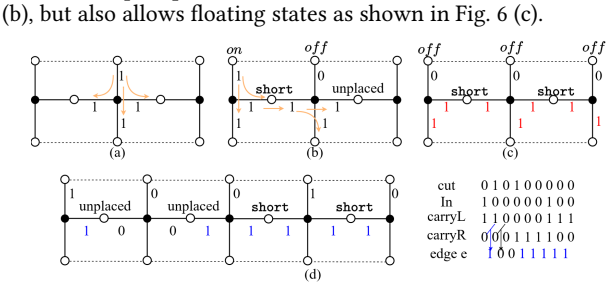

Figure 6: Flow–propagation examples. (a) When the $y_{k,e^{\uparrow}(v)}$ is 1, incident edges mirror. (b) When the $y_{k,e^{\uparrow}(v)}$ is 0, left/right/bottom may still become 1, motivating the second clause of Eq. (2). (c) An undesirable floating case that Eq. (2) alone cannot forbid. (d) Floating elimination by introducing left–to–right / right–to–left carry variables that enforce segmentwise coherence.

4.3.3 ***Floating elimination.*** To prevent floating (horizontal edges set to 1 without any sources), we activate a horizontal edge only if it belongs to a contiguous `short` that contains a real inflow seed. This eliminates cases as shown in Fig. 6 (c). Along an odd row r (columns c=0, . . . , ℓ-1), let $e_{r,c}$ denote the horizontal edge between positions c and c+1. Define a gap indicator cut_c that is 1 iff `short` placed at column c; otherwise 0. Define horizontal carries:

$$\begin{aligned}
\mathrm{carryL}_0 &= \mathrm{In}_k(r,0), \quad \mathrm{carryR}_\ell = \mathrm{In}_k(r,\ell),\\
\mathrm{carryL}_c &= (\neg\mathrm{cut}_{c\text{-}1} \wedge \mathrm{carryL}_{c\text{-}1}) \vee \mathrm{In}_k(r,c) \qquad \text{for } c = 1,\ldots,\ell,\\
\mathrm{carryR}_c &= (\neg\mathrm{cut}_{c+1} \wedge \mathrm{carryR}_{c+1}) \vee \mathrm{In}_k(r,c) \qquad \text{for } c = 0,\ldots,\ell\text{-}1.
\end{aligned}$$

Then each horizontal edge flow variable is enabled iff the segment carries inflow from either side:

$$y_{k,e_{r,c}} \Leftrightarrow \mathrm{carryL}_{c+1} \vee \mathrm{carryR}_c, \qquad c = 0,\ldots,\ell\text{-}1.$$

4.3.4 ***Placement and structural constraints.*** We restrict placements to avoid ambiguity and prune spurious search.
Row coverage: Every even row r_{even} has at least one placement providing the minimal condition for a $vdd \to f$ path.

$$\forall r \in r_{\text{even}} : \bigvee_{v \in r} \bigvee_{p \in (X \cup O^p \cup \{\texttt{short}\})} x_{p,v}.$$

Anti-dangling: For each S/D node $v_{s/d}$ with neighbor set $N(v_{s/d})$, define the adjacent placement literals as $\mathcal{A}(v_{s/d}) = \{\, x_{p,v} \mid v \in N(v_{s/d}),\ p \in X \cup O^p \cup \{\texttt{short}\} \,\}$. We enforce the anti-dangling condition as stated in Eq. (3).

$$\sum_{adj \in \mathcal{A}(v_{s/d})} adj \neq 1 \quad (3) \qquad\qquad \min \sum_{v} x_{\texttt{short},v} \quad (4)$$

4.3.5 ***Objective (lexicographic).*** We first minimize Eq. (1); subject to this, we minimize the number of `short` placements by Eq. (4), leaving only those required to ensure a $vdd \to f$ path.

4.4 Complement-Pair Derivation

Traditional transistor network synthesis either synthesizes PMOS and NMOS independently [19] or simultaneously [17]. We propose a complement-pair derivation technique that exploits the planar dual [18] or applies a complement-polarity transform, then combines NMOS candidates from multiple PMOS solutions to assemble P/N pairs—reducing synthesis effort and improving layout quality.

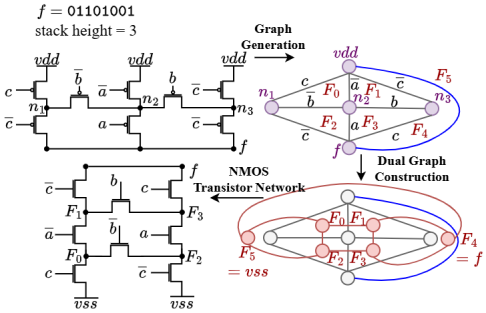

Figure 7: Planar-dual graph generation for f=01101001.

1) Graph Analysis. Figure 7 illustrates the dual-graph generation procedure. We view a transistor network as an S/D connectivity graph G: vertices are S/D nets, edges are transistors, and each edge is labeled by its gate net. Accordingly, we write G_p for the PMOS pull–up graph and G_n for the NMOS pull–down graph. **Step 1: Planarity with an auxiliary edge.** To split the unbounded outer face and identify the dual terminals, we add an auxiliary edge (vdd, f) and run a linear-time planarity test (Boyer–Myrvold [3]) on $G_p \cup \{(vdd, f)\}$. **Step 2: Dual graph construction.** If the graph admits a planar embedding, we construct its dual as follows: each face F becomes a dual vertex, and for every primal edge e with adjacent faces $F_L(e)$ and $F_R(e)$ we add a dual edge $(F_L(e), F_R(e))$, copying the gate label of e onto the dual edge. The two faces adjacent to the auxiliary edge are designated as vss and f, respectively. **Step 3: Stack-height bound via polarity-consistent paths.** We compute the longest path between vss and f in G_n: edge labels appear along the path with at most one polarity (paths mixing x and $\bar{x}$ are pruned). The path length determines the NMOS stack height, M_n. The M_n in Fig. 7 is 4: the longest path with 5 transistors is pruned ($b \Leftrightarrow \bar{b}$). **2) NMOS candidates generation.** By leveraging multiple PMOS solutions, we can derive multiple NMOS candidates for layout optimization. **Strategy 1: dual graph.** If a PMOS solution is planar and its dual graph satisfies stack height $M_n \leq 4$, we add the planar dual G_n as an NMOS candidate. **Strategy 2: complement-polarity transform.** We can derive an additional NMOS network via complement–polarity transform (Algorithm 2).

Algorithm 2 Complement-Polarity Transform

Input: Boolean function f, synthesized PMOS library L_P.
Output: Set of NMOS networks $S_n^\star$ for f.

```
f̄ ← ComplementOutput(f)                    ▷ e.g., f = 0011 ⇒ f̄ = 1100
f_eq ← LookupPClass(f̄, L_P)                ▷ f_eq: f̄ or its P-equivalent in L_P
S_n ← L_P[f_eq].solutions
S_n^⋆ ← ∅
for s_n ∈ S_n do
    s_n^⋆ ← PolarityTransform(s_n)         ▷ Input polarity transform (e.g., x ↦ x̄)
    Append(S_n^⋆, s_n^⋆)
end for
return S_n⋆
```

First, take the truth-table complement: $\bar{f} \leftarrow \textsc{ComplementOutput}(f)$, so $\bar{f}(\mathbf{k})$=1-$f(\mathbf{k})$ (e.g., `0011` $\Rightarrow$ `1100`). Next, query the PMOS library L_P for a P–class representative of $\bar{f}$: $f_{\text{eq}} \leftarrow \textsc{LookupPClass}(\bar{f}, L_P)$. If an exact match exists, return it; otherwise, return a P-equivalent representative. Let S_n be the PMOS solutions in L_P for f_{eq}. For each $s_n \in S_n$, obtain an NMOS candidate $s_n^\star$ by inverting all input polarities (e.g., $x \mapsto \bar{x}$). Finally, we form P/N combinations by pairing each PMOS solution with NMOS candidates.

5 Layout: Cell Library Optimization

We co-optimize circuit and layout by enumerating P/N pairs, evaluating each across multiple layout architectures, selecting the top candidate per metric, and classifying them into two objective-oriented libraries. Our layout metrics are #CPP, pin accessibility, and metal usage (#M1/#M2). While SO3-Cell [5] already incorporates circuit-topology optimization, it is limited to single-height (SH). Our 3-input two-stage cells exhibit strong inter-stage signal sharing, so we extend the ILP framework to double-height (DH) circuit-topology optimization, enumerating and selecting topologies that best exploit inter-row signal sharing and reduce metal usage. We consider three architectures: SH, DH with rows meeting at the PMOS side ($\mathrm{DH}_{\mathrm{vss}}$), and DH with rows sharing the NMOS side ($\mathrm{DH}_{\mathrm{vdd}}$). We build two objective-driven libraries to analyze the area–routability trade-off: (i) an *Area-Optimized* set ($\mathrm{L2L}_{\mathrm{AO}}$), selecting layouts with minimal #CPP; and (ii) a *Metal-Usage-Optimized* set ($\mathrm{L2L}_{\mathrm{MO}}$), selecting layouts with reduced metal usage.

6 Experiments

All experiments are run on a Linux workstation with an Intel Xeon E5–2620 (2.40 GHz) and 256 GB RAM. For logic and circuit exploration, we use Z3 *v4.15.3*; the maximum stack height is fixed at 4. This reflects a common practice [15] where increasing the number of stacked transistors beyond 4 severely degrades the on-current. For layout exploration, we implement SO3-Cell [5], using

the Gurobi Optimizer *v12.0.1* via its Python API. We run LVS/PEX with *Cadence Pegasus v21.3* and characterize at (TT, 0.7 V, 25°C) using *Cadence Liberate v23.1* under the Non-Linear Delay Model (NLDM). Finally, we evaluate the block PPA improvement in the baseline (prior best SO3-Cell L_3) after augmenting it with our new libraries ($L2L_{AO}$/$L2L_{MO}$) in *Synopsys Fusion Compiler v22.12*.

6.1 Logic Level Exploration

6.1.1 One-Stage Synthesis. Table 1 summarizes the permutation-closed input pool O^P. Each row groups representative one-stage functions O obtained from one-stage synthesis for 78 three-input P-class functions (excluding the always–0/1 pair). The column "#" reports the number of distinct P-equivalent embeddings, yielding 18 functions in O^P. Together with the 3 primaries, we utilize 21 inputs for the full two-stage synthesis.

Table 1: Distinct P-equivalent instantiations (on $X = a, b, c$) for O^P; "#" indicates embeddings per base function.

Function Category	Representative base function(s)	#
1-input INV	$\neg a$	3
2-input NAND / NOR	$\neg(a \wedge b)$, $\neg(a \vee b)$	3
3-input NAND / NOR	$\neg(a \wedge b \wedge c)$, $\neg(a \vee b \vee c)$	1
3-input AOI21 / OAI21	$\neg\big((a \wedge b) \vee c\big)$, $\neg\big((a \vee b) \wedge c\big)$	3
3-input minority	$\neg\big((a \vee b) \wedge (a \vee c) \wedge (b \vee c)\big)$	1

6.1.2 Two-Stage Synthesis. Table 2 summarizes *Connected–Diffusion* (arbitrary–topology) results. With the same input pool as prior

Table 2: *Connected–Diffusion* results for 3-input P-class.

Input Pool	Stack Height (M)	Coverage	Transistor (PMOS)
X	4	10%	N/A
$X \cup \bar{X}$	2	59%	N/A
$X \cup \bar{X}$	3	100%	515
$X \cup \bar{X}$	4	100%	515
$X \cup O^P$	2	74%	N/A
$X \cup O^P$	3	100%	442
$X \cup O^P$	4	100%	442

work [19], we obtain a total of 515 PMOS transistors with M equals 3 and 4. Using the permutation-closed pool O^P with $M \in \{3, 4\}$ reduces the aggregate to 442 while covering all 3-input functions; 6 functions admit multiple cost-optimal solutions. With M=2, 10 functions are unrealizable and 7 realizable ones need +1 transistor versus their best. Figure 8 illustrates how stack–height bounds and

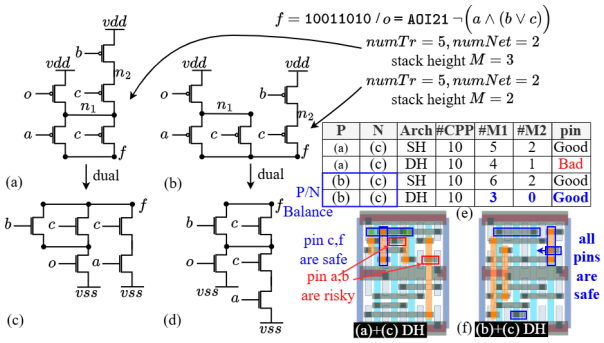

Figure 8: Layout generation process for f = 10011010. (a-b) PMOS solution with stack height 3 and 2. (c–d) Corresponding NMOS networks obtained via planar dual. (e) Layout metrics comparing (a)+(c) vs. (b)+(c). (f) DH layouts: Pins in (a)+(c) show poorer accessibility, whereas the pins in (b)+(c) allow greater stretch-and-route flexibility.

multiple solutions can be leveraged. With a stack-height bound of 3, we obtain the PMOS solution in (a); tightening the bound to 2 yields (b). Applying the planar dual yields the corresponding NMOS: (c) with stack height 2 and (d) with stack height 3. Pairing (b) with (c) thus balances P/N stack heights (beneficial for circuit performance) and improves layout quality, as reflected in the metrics summarized in (e). The pin metric in (e) quantifies pin accessibility of the cell layout. Figure 8 (f) compares DH implementations of the pairings (a)+(c) and (b)+(c). In (a)+(c), pins b and a suffer from poor accessibility, whereas the balanced 2-stack in (b)+(c) achieves better pin accessibility by leveraging the available empty metal track.

6.2 Circuit Level Exploration

6.2.1 Arbitrary vs Series-Parallel. The *Connected–Diffusion* (arbitrary) yields a smaller transistor count than the *Grid–Scaffold* (*series–parallel*) for 4 functions. Table 3 shows the total transistors

Table 3: Comparison of transistor count and layout area for 4 functions whose series–parallel and arbitrary implementations differ in transistor count. Areas (#CPP) are reported for SH, DH, and the ideal (ideal #CPP = # of transistors / 2 + 1).

f	*series–parallel* (Total **896** transistors)		arbitrary (Total **884** transistors)	
	# of transistors	Area (SH/DH/ideal)	# of transistors	Area (SH/DH/ideal)
f_0=01101001	26 (+4)	X / 16 / 14	22	X / X / 12
f_1=10000110	18 (+2)	10 / 12 / 10	16	X / 12 / 9
f_2=10010110	24 (+4)	X / 14 / 13	20	X / 12 / 11
f_3=10011110	18 (+2)	10 / 12 / 10	16	X / 12 / 9

X indicates infeasible within the stated budget: In the *series–parallel*, failure to meet ideal+2CPP; In the arbitrary, failure to achieve #CPP no larger than the *series–parallel*.

difference (*series-parallel* 896 vs arbitrary 884), lists 4 functions with different transistor count and compares layout realizations in terms of #CPP for SH, DH, and ideal. Only f_2 carries its 4-transistor reduction through to a layout gain; for the other 3 functions, no feasible layout improves upon the *series–parallel* result. Three functions f_1, f_2, f_3 all converge to a common planar non–series–parallel topology. Figure 9 (a) depicts the shared second-stage backbone,

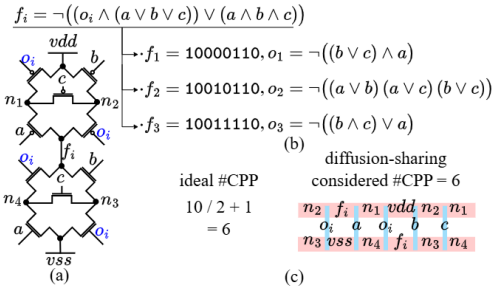

Figure 9: Planar topology (a) is the common second-stage backbone for f_1, f_2, f_3 in Table 3. (b) depicts their distinct first-stage functions o_1, o_2, o_3; (c) shows two ways to estimate #CPP.

and Figure 9 (b) shows how each function realizes o_i via different first-stage outputs o_1, o_2, o_3. Figure 9 (c) suggests that both the ideal and the diffusion–sharing considered #CPP is 6. However, the 4-track SH layout (Fig. 10 (a)) of Figure 9 (a) requires +2 dummy due to routing complexity. By contrast, the DH implementation (Fig. 10 (b)) of f_2—placing the first stage o_2 in the 1st row and the second stage in the 2nd row—enables inter-row gate sharing and eliminates the need for dummies. As a result, the 4-transistor reduction for f_2 translates into layout gains (-2 CPP).

6.2.2 DH & Topology Optimization. Figures 11 (a-b) are logically equivalent circuits for f=01101000 with different device connections, and (c-d) show their corresponding DH layouts. When mapped to DH layouts, circuit topology (b) enables more inter-row gate sharing that reduces M1 usage by 3 tracks compared to (a). Since our layout framework performs circuit-topology optimization across

all architectures, it yields objective-function–optimal topologies within each architecture, regardless of the initial input circuit.

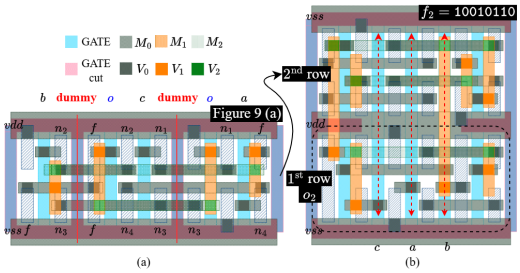

Figure 10: (a) shows the SH layout of Fig. 9 (a), requiring two extra dummy CPPs beyond the estimate in Fig. 9 (c). In (b), first-stage logic o_2 enables inter-row gate sharing, eliminating two dummy CPPs in the DH layout of f_2 = 10010110.

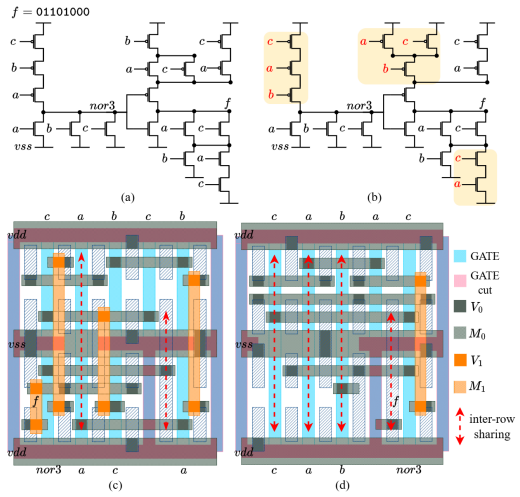

Figure 11: (a-b) are logically equivalent circuits for f=01101000, and (c-d) their DH implementations. Changing (a) to (b) improves the DH layout, reducing M1 usage by 3 tracks through inter-row gate sharing.

6.3 Layout Level Exploration

We generate 64 cell layouts out of the 78 three–input P-class, as 14 are already available in the reference library. Among the 64 generated cells, 32 layouts are included in both L2L$_{AO}$ and L2L$_{MO}$ because a single implementation achieves both the minimum #CPP and the lowest metal usage. For the remaining 32 cells, we provide two distinct variants, each aligned with its respective library's objective. DH optimization typically trades a #CPP increase for reducing metal usage; all metal-efficient selections in L2L$_{MO}$ are DH implementations, which on average save about 3 M1 tracks and 1–2 M2 tracks at a cost of roughly +1 CPP [1]. Figure 12 illustrates a layout example for f = 11101001. The SH implementation achieves the minimum area (11 CPP) and is selected for L2L$_{AO}$. The DH$_{vdd}$ implementation reduces metal usage by 4 M1 and 2 M2 tracks at the cost of +1 CPP, and is therefore selected for L2L$_{MO}$. The complete set of finalized layouts and per-cell statistics (Cell Name, f, #CPP, #M1, #M2) is publicly available in the open-source L2L library [1].

6.4 Block Experiments

We use three RTL designs from OpenCores [11]. We minimize block area subject to: worst negative slack < 20 ps, design rule violations < 200, and shorts < 20. Frequency is reported as the reciprocal

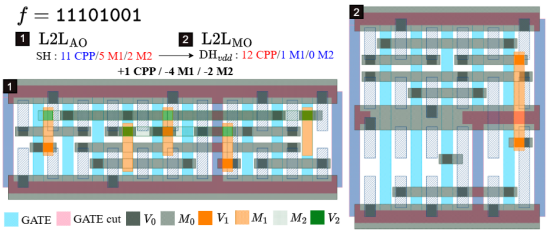

Figure 12: Two layout implementations for f = 11101001

Table 4: Block-level comparison of the best prior SO3-Cell L_3 versus L_3+L2L$_{AO}$ and L_3+L2L$_{MO}$. (power (mW), frequency (GHz), and area (μm^2)).

Library		Metrics	AES - 530 FFs		JPEG - 4454 FFs		VGA - 17041 FFs		*Norm.Avg.*	
Name	SO3-Cell L_3	Power	4.53	0%	4.36	0%	20.29	0%	1.00	0%
		Freq.	7.74	0%	3.06	0%	6.03	0%	1.00	0%
# of cells	51	Area	415.72	0%	1295.66	0%	2883.03	0%	1.00	0%
Name	L_3+**L2L$_{AO}$**	Power	4.48	-1.1%	4.40	+0.9%	20.28	+0.0%	**1.00**	-0.1%
		Freq.	7.45	-3.7%	3.21	+5.0%	5.99	-0.7%	**1.00**	+0.2%
# of cells	115	Area	399.73	-3.8%	1280.81	-1.1%	2878.21	-0.2%	**0.98**	-1.7%
Name	L_3+**L2L$_{MO}$**	Power	4.47	**-1.3%**	4.32	-1.0%	20.24	-0.2%	**0.99**	-0.8%
		Freq.	7.47	-3.5%	3.23	**+5.6%**	6.01	-0.3%	**1.01**	+0.6%
# of cells	115	Area	398.83	**-4.1%**	1252.62	-3.3%	2868.07	-0.5%	**0.97**	-2.6%

of the effective clock period, defined as the target clock minus the average slack of the 100 worst paths. Table 4 shows a block-level comparison. Overall, L_3+L2L$_{MO}$ achieves better PPA than both the baseline SO3-Cell L_3 and L_3+L2L$_{AO}$, with −0.8% power, +0.6% performance, and −2.6% area, while L2L$_{AO}$ attains about −0.1%, +0.2%, and −1.7%, respectively. The benefit is design-dependent: VGA, being FF-dominant, shows limited gain, whereas AES and JPEG see area reductions of −4.1% and −3.3% with L2L$_{MO}$. Figure 13

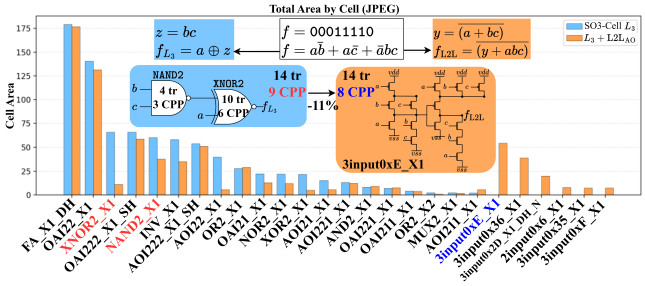

Figure 13: Highlighted is 3input0xE_X1 (f=00011110), which occupies the largest area among added 3-input P-class cells and is 11% smaller than the NAND2+XOR2 realization.

reports the per-cell area in the JPEG block and illustrates how 3-input cells are utilized. With L2L$_{AO}$ enabled, the most frequent 3-input instance is 3input0xE_X1 (f=00011110). The Boolean simplification in Fig. 13 shows that this function can be composed from XNOR and NAND2; correspondingly, after adding L2L$_{AO}$, we observe reduced area of NAND2 and XOR2 cells. Because 3input0xE_X1 is 11% smaller than the NAND2+XOR2, the commercial tool—accounting for wirelength and other PPA factors—naturally exploits this gain.

7 Conclusion

In this work, we present an end-to-end L2L framework that explores and optimizes the logic, circuit, and layout stages, delivering block-level PPA gains. While generating a physical cell library for the full 3-input P-class (80 functions) is manageable, scaling to all 4-input P-class functions (3,984) and beyond is expensive; as future work, we will pursue demand-driven, RTL-aware discovery of high-leverage cells rather than exhaustive generation.

References

[1] [n. d.]. L2L. https://github.com/b8kang/L2L GitHub repository.

[2] Kyeonghyeon Baek and Taewhan Kim. 2021. Simultaneous Transistor Folding and Placement in Standard Cell Layout Synthesis. In *Proc. ICCAD*. 1–8. https://doi.org/10.1109/ICCAD51958.2021.9643537

[3] John M. Boyer and Wendy J. Myrvold. 2004. On the Cutting Edge: Simplified O(n) Planarity Algorithms. *J. Graph Algorithms Appl.* 8, 3 (2004), 241–273. https://doi.org/10.7155/jgaa.00091

[4] Bilal Chehab, Odysseas Zografos, Eugenio Dentoni Litta, Zubair Ahmed, Pieter Schuddinck, Doyoung Jang, Geert Hellings, Alessio Spessot, Pieter Weckx, and Julien Ryckaert. 2021. Two-level MOL and VHV routing style to enable extreme height scaling beyond 2nm technology node. In *Proc. IITC*. 1–3. https://doi.org/10.1109/IITC51362.2021.9537557

[5] Chung-Kuan Cheng, Andrew B Kahng, Byeonggon Kang, Seokhyeong Kang, Jakang Lee, and Bill Lin. 2025. SO3-Cell: Standard Cell Layout Automation Framework for Simultaneous Optimization of Topology, Placement, and Routing. In *Proc. ICCAD*. https://doi.org/10.1109/ICCAD66269.2025.11240677

[6] Chung-Kuan Cheng, Byeonggon Kang, Bill Lin, and Yucheng Wang. 2025. Standard Cell Layout Generation: Review, Challenges, and Future Works. In *Proc. ASP-DAC*. 372–378. https://doi.org/10.1145/3658617.3703146

[7] Sehyeon Chung, Hyunbae Seo, Handong Cho, Kyumyung Choi, and Taewhan Kim. 2024. Optimal Layout Synthesis of Multi-Row Standard Cells for Advanced Technology Nodes. In *Proc. ICCAD*. 1–8. https://doi.org/10.1145/3676536.3676784

[8] Winston Haaswijk, Eleonora Testa, Mathias Soeken, and Giovanni De Micheli. 2017. Classifying functions with exact synthesis. In *Proc. ISMVL*. 272–277. https://doi.org/10.1109/ISMVL.2017.44

[9] Byeonggon Kang, Ying Yuan, Yucheng Wang, Bill Lin, and Chung-Kuan Cheng. 2025. Cell-Flex Metrics for Designing Optimal Standard Cell Layout with Enhanced Cell Layout Flexibility. In *Proc. ISPD*. 164–172. https://doi.org/10.1145/3698364.3705344

[10] Daeyeal Lee, Dongwon Park, Chia-Tung Ho, Ilgweon Kang, Hayoung Kim, Sicun Gao, Bill Lin, and Chung-Kuan Cheng. 2020. SP&R: SMT-based simultaneous Place-and-Route for standard cell synthesis of advanced nodes. *IEEE Trans. on CAD* 40, 10 (2020), 2142–2155. https://doi.org/10.1109/TCAD.2020.3037885

[11] OpenCores. [n. d.]. Open-Source IP Cores. https://opencores.org/

[12] Vinicius N. Possani, Vinicius Callegaro, André I. Reis, Renato P. Ribas, Felipe de Souza Marques, and Jr. Leomar S. da Rosa. 2016. Graph-Based Transistor Network Generation Method for Supergate Design. *IEEE Trans. on VLSI* 24, 2 (2016), 692–703. https://doi.org/10.1109/TVLSI.2015.2410764

[13] Vinicius Neves Possani, Vinicius Callegaro, André Inácio Reis, Renato Perez Ribas, Felipe de Souza Marques, and Leomar Soares da Rosa Junior. 2013. Efficient transistor-level design of CMOS gates. In *Proc. GLSVLSI*. 191–196. https://doi.org/10.1145/2483028.2483089

[14] Haoxing Ren and Matthew Fojtik. 2021. Nvcell: Standard cell layout in advanced technology nodes with reinforcement learning. In *Proc. DAC*. 1291–1294. https://doi.org/10.1109/DAC18074.2021.9586188

[15] Nikhil Saxena and Sonal Soni. 2013. Leakage current reduction in CMOS circuits using stacking effect. *Int. J. Appl. Innov. Eng. Manag.* 2, 11 (2013), 213–216.

[16] Jaewoo Seo, Jinwook Jung, Sangmin Kim, and Youngsoo Shin. 2017. Pin accessibility-driven cell layout redesign and placement optimization. In *Proc. DAC*. 1–6. https://doi.org/10.1145/3061639.3062302

[17] Jiun-Cheng Tsai, Wei-Min Hsu, Kuei-Lin Wu, Hsuan-Ming Huang, Jen-Hang Yang, Heng-Liang Huang, Yen-Ju Su, and Charles H.-P. Wen. 2025. MuSTNet: SAT-based Exact Multi-Stage Transistor Network Synthesis with Placement Awareness. In *Proc. ICCAD*. https://doi.org/10.1109/ICCAD66269.2025.11240651

[18] Hassler Whitney. 1933. Planar Graphs. In *Hassler Whitney: Collected Papers*. Springer, 135–146. https://doi.org/10.1007/978-1-4612-2972-8_9

[19] Weihua Xiao, Shanshan Han, Yue Yang, Shaoze Yang, Cheng Zheng, Jingsong Chen, Tingyuan Liang, Lei Li, and Weikang Qian. 2023. MiniTNtk: An Exact Synthesis-based Method for Minimizing Transistor Network. In *Proc. ICCAD*. 01–09. https://doi.org/10.1109/ICCAD57390.2023.10323691